edureka!

MASTERING PYTHON WITH EDUREKA

A Guide to Learn Python Programming

Don't just Learn it, Master it!

TABLE OF CONTENTS

1. INTRODUCTION TO PYTHON 3

2. PYTHON INSTALLATION 5

3. PYTHON FUNDAMENTALS 6

- Keywords & Identifiers
- Variables
- Comments
- Operators
- Functions

4. DATA TYPES IN PYTHON 8

- Dictionary
- Lists
- Tuple
- Set

5. FLOW OF CONTROL IN PYTHON 10

- Iterative Statements
- Conditional Statements

TABLE OF CONTENTS

6. PYTHON OOPS CONCEPTS 12

- Classes & Objects
- Abstraction
- Encapsulation
- Inheritance
- Polymorphism

7. PYTHON PRACTICE PROGRAMS 15

8. TOP 40 INTERVIEW QUESTIONS 16

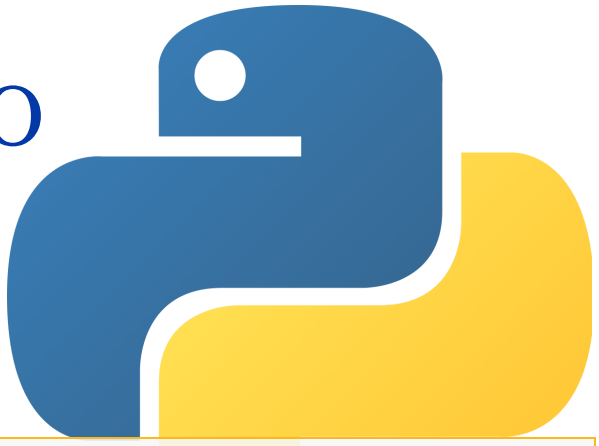
9. CAREER GUIDANCE 18

- How to become a Python Developer?
- Edureka's Structured Training Programs

10. REFERENCES 19

Chapter 1

INTRODUCTION TO PYTHON PROGRAMMING



Programming languages have been around for ages, and every decade sees the launch of a new language sweeping developers off their feet. Python is considered as one of the most popular and in-demand programming languages. A recent Stack Overflow survey showed that Python has taken over languages such as Java, C, C++ and has made its way to the top. This makes Python Programming one of the most sought-after programming languages across the globe.

GET READY TO FALL IN LOVE WITH **PYTHON PROGRAMMING**

1.1 What is Python Programming?

Python is a high-level, interpreted language that has an easy syntax and dynamic semantics. Python is much easier than other programming languages and helps you create beautiful applications with less effort and much more ease. Since its inception in the 1990s, Python has become hugely popular and even today there are thousands who are learning this Object-Oriented Programming language.

PYTHON FEATURES

Easy to Learn & Use

Free and Open Source

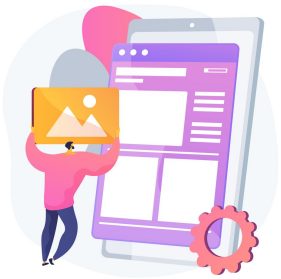
High-Level Language

Highly Portable

Object-Oriented Programming

Comprehensive Set of Libraries

PYTHON APPLICATIONS



GUI Based Applications

Game Development

Prototyping

Web Servers Programming

Data Science and Machine Learning

Chapter 2

PYTHON INSTALLATION

2.1 Downloading Python

1. Go to www.python.org/downloads/
2. Download Python as per your system requirement

INSTALLING PYTHON ON WINDOWS

1. Click on **Python Releases** for Windows, select the link for the latest Python 3 Release – Python 3.x.x
2. Scroll to the bottom and select either **Windows x86-64** executable installer for 64-bit or Windows x86 executable installer for 32-bit

INSTALLING PYTHON ON LINUX

1. Open the Ubuntu Software Center folder
2. Select **Developer Tools** from the All Software drop-down list box
3. Double-click the **Python 3.3.4** entry
4. Click **Install**
5. Close the Ubuntu Software Center folder

2.2 Best Python IDEs

IDE stands for Integrated Development Environment which is a Graphical User Interface where programmers write their code to produce the final products. An IDE basically unifies all essential tools required for software development and testing. In order to make the best use of this e-book, install an IDE now and start implementing the Python concepts as you learn.

Always keep the following points in mind while choosing the best IDE for Python:

- Level of expertise of the programmer
- The type of industry or sector where Python is being used
- Ability to buy commercial versions or stick to the free ones
- Kind of software being developed
- Integration with other languages

We have listed the Top 5 Python IDE's chosen by the Python Enthusiasts →

Jupyter Notebook

PyCharm

Spyder

PyDev

Atom

Chapter 3

PYTHON FUNDAMENTALS

3.1 Keywords and Identifiers

Keywords are nothing but special names that are already present in python. We can use these keywords for specific functionality while writing a python program.

```
#retrieving all keywords
import keyword
keyword.kwlist
#
keyword.iskeyword('try')
#this will return true, if the mentioned name is a keyword
```

Identifiers are user-defined names that we use to represent variables, classes, functions, modules, etc.

```
name = 'edureka'
my_identifier = name

#this will return the value of the identifier provided by the user
```

3.2 Variables

Variables are like a memory location where you can store a value. This value, you may or may not change in the future.

```
x = 10
y = 20
name = 'edureka'

#To declare a Python variable you only have to assign a value to it
```

3.3 Comments

Comments in programming are the programmer-coherent statements, that describe what a block of code means. They are very useful when you are writing large codes. Comments in Python start with a # character. Alternatively, at times, commenting is done using docstrings (strings enclosed within triple quotes). Python Comments can be of two types:

- Single-line or
- Multi-line

```
#Comments in Python start like this
print("Comments in Python start with a #")
```

3.4 Operators

Operators in Python are used for operations between two values or variables. The output varies according to the type of operator used in the operation. We can call operators as special symbols or constructs to manipulate the values of the operands. Consider the expression 2 + 3 = 5, here 2 and 3 are operands and + is called operator.

Type	Operators
Arithmetic	+, -, *, /, %, **, //
Assignment	=, +=, -=, *=, %=, **=, //=, |=, ^=, &=
Comparison	==, !=, >, <, <=,>=
Logical	and, or, not
Membership	in, not in
Identity	is, is not
Bitwise	&, |, ^, ~, <<,>>

3.5 Functions

A function in Python is a block of code that will execute whenever it is called. We can pass parameters in the functions as well. To understand the concept of functions, let's take an example.

Suppose you want to calculate the factorial of a number. You can do this by simply executing the logic to calculate a factorial. But what if you have to do it ten times in a day, writing the same logic again and again is going to be a long task. Instead, what you can do is, write the logic in a function. Call that function every time you need to calculate the factorial. This will reduce the complexity of your code and save your time as well.

```
#declaring a function
def function_name():
#expression
print('abc')
```

```
#calling a function
def my_func():
print('function created')
#this is a function call
my_func()
```

Function Parameters

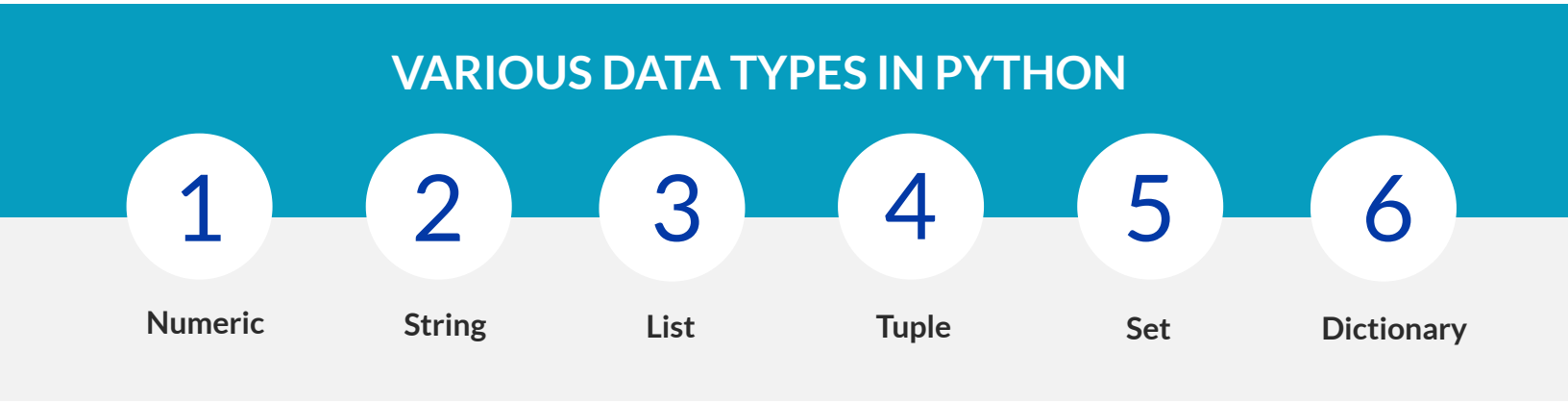
We can pass values in a function using the parameters. We can use also give default values for a parameter in a function as well.

```
#
def my_func(name = 'edureka'):
    print(name)

#default parameter
 my_func()

#userdefined parameter
my_func('python')
```

Chapter 4

DATA TYPES IN PYTHON

Variables are used to hold values for different data types. As Python is a dynamically typed language, you don't need to define the type of the variable while declaring it. The interpreter implicitly binds the value with its type. Python enables us to check the type of the variable used in the program. With the help of the type() function, you can find out the type of the variable passed.



4.1 Numeric

Numerical data type holds numerical value. In numerical data, there are 4 subtypes as well. Following are the sub-types of numerical data type:

a. ***Integers*** *- Integers are used to represent whole number values*

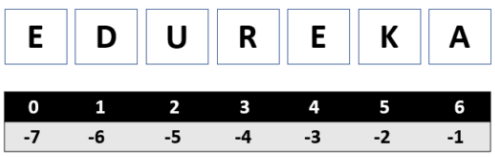
b. ***Float*** *- Float data type is used to represent decimal point values*

c. ***Complex Numbers*** *- Complex numbers are used to represent imaginary values*

d. ***Boolean*** *- Boolean is used for categorical output, since the output of boolean is either true or false*

4.2 String

Strings in Python are used to represent Unicode character values. Python does not have a character data type, a single character is also considered as a string. We declare the string values within single quotes or double-quotes. Indexes and square brackets are used to access the values. Strings are immutable in nature.




```
name = 'edureka'
name[2]
#this will give you the output as 'u'

name = 'edureka'
name.upper()
#this will make the letters to
uppercase
name.lower()
#this will make the letters to
lowercase
name.replace('e') = 'E'
#this will replace the letter 'e' with
'E'
name[1: 4]
#this will return the strings starting
```

4.3 List

List is one of the four collection data type that we have in python. When we are choosing a collection type, it is important to understand the functionality and limitations of the collection. Tuple, set and dictionary are the other collection data type in Python. A list is ordered and changeable, unlike strings. We can add duplicate values as well. To declare a list, we use the square brackets.

```
mylist = [10,20,30,40,20,30, 'edu']
mylist[2:6]
#this will get the values from index 2 to
6
[6] = 'python'
#this will replace the value at the index
6
mylist.append('edureka')
#this will add the value at the end
mylist.insert(5, 'data science')
#this will add the value at the index 5
```

4.4 Tuple

Tuple is an ordered data structure whose values can be accessed using the index values. It can have duplicate values. To declare a tuple, we use the round brackets. A tuple is a read-only data structure and you cannot modify the size and value of the items of a tuple.

```
#declaring o tuple
mytuple = (10,10, 20, 30, 40, 50)
#counting total number of elements
mytuple.count(10)
#to find an item index
mytuple.index(50) #output will be 5
```

4.5 Set

A set is a collection that is unordered & doesn't have any index. To declare sets in Python, we use curly brackets. A set does not have any duplicate values. Even though it will not show any errors while declaring the set, the output will only have distinct values.

```
myset = { 10, 20 , 30 , 40, 50, 50}
#to add a value in a set
myset.add('edureka')
#to add multiple values in a list
myset.update([ 10, 20, 30, 40, 50])
#to remove an item from a set
myset.remove('edureka')
```

4.6 Dictionary

A dictionary is just like any other collection array in Python. But they have key-value pairs. A dictionary is unordered and changeable. We use the keys to access the items from a dictionary. To declare a dictionary, we use curly brackets.

```
mydictionary = { 'python': 'data science',
'machine learning' : 'tensorflow' ,
'artificial intelligence': 'keras'}

mydictionary['machine learning']
#this will give the output as 'tensorflow'

mydictionary.get('python')
#this serves the same purpose to access
the value
```

Chapter 5

FLOW OF CONTROL

*Code runs sequentially in any language, but what if you want to break that flow such that you are able to add logic and repeat certain statements such that your code reduces and are able to obtain a solution with lesser and smarter code. After all, that is what coding is. Finding logic and solutions to problems and can be done using **Conditional** and **Iterative** statements.*

5.1 Conditional Statements

Conditional statements are executed only if a certain condition is met, else it is skipped ahead to where the condition is satisfied. There are various types of conditional statements supported in Python.

IF

An if statement is used to test an expression and execute certain statements accordingly. A program can have many if statements.

ELSE

An else statement is used with an if statement. Else contains the block of a code that executes if the conditional expression in the 'if statement' is FALSE.

ELIF

The elif statement allows a number of expression checks for TRUE and execute a block of code as soon as one of the conditions returns TRUE

```
if condition:statement
elif condition:statement
else:
statement
```

This means that if a condition is met, do something. Else go through the remaining elif conditions and finally if no condition is met, execute the else block. You can even have nested if-else statements inside the if-else blocks.

5.2 Example

```
a = 10
b = 15
if a == b:
    print ( 'They are equal' )
elif a > b:
    print ( 'a is larger' )
else :
    print ( 'b is larger' )
```

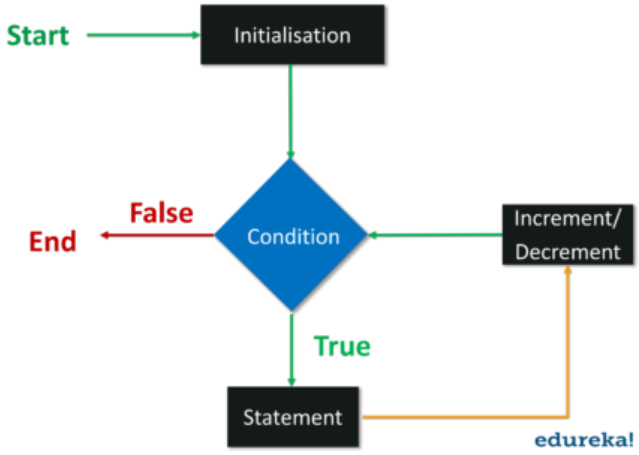
OUTPUT: b is larger

5.2 Iterative Statements

Loops in Python allow us to execute a group of statements several times. Loops can be divided into 2 kinds:

a. **Finite:** This kind of loop works until a certain condition is met

b. **Infinite:** This kind of loop works infinitely and does not stop ever



Loops in Python or any other language have to test the condition and they can be done either before the statements or after the statements. They are called:

a. **Pre-Test Loops**: Where the condition is tested first and statements are executed subsequently

b. **Post Test Loops**: Where the statement is executed at least once and later the condition is checked

FOR

This loop is used to perform a certain set of statements for a given condition and continue until the condition has failed

```
#syntax
for variable in range: statements
#example
fruitsBasket= ['apple', 'orange', 'pineapple',
'banana']
for fruit in fruitsBasket:
    print(fruit, end=',')
#Output is apple, orange, pineapple, banana
```

WHILE

This loop in Python is used to iterate over a block of code or statements as long as the test expression is true.

```
#syntax
while (test expression): statements
#example
second = 5
while second >= 0:
    print(second, end='->')
    second-=1
print('Blastoff!')
#Output is 5->4->3->2->1->Blastoff!
```

Chapter 6

OBJECT-ORIENTED PROGRAMMING IN PYTHON

Object-Oriented Programming (OOP) is a way of computer programming using the idea of "objects" to represent data and methods. It is also an approach used for creating neat and reusable code instead of a redundant one. The program is divided into self-contained objects or several mini-programs. Every individual object represents a different part of the application having its own logic and data to communicate within itself.

6.1 Classes & Objects

Class is a collection of objects or you can say it is a blueprint of objects defining the common attributes and behavior. It logically groups the data in such a way that code reusability becomes easy. Class is defined under a "Class" keyword. Using a Class, you can add consistency to your programs so that they can be used in an efficient way. The attributes of a Class are listed below:

a. **Class variable** is a variable that is shared by all the different objects/instances of a class

b. **Instance variables** are variables that are unique to each instance. It is defined inside a method and belongs only to the current instance of a class

c. **Methods** are also called functions that are defined in a class and describe the behavior of an object

```
#syntax
class EduClass():
```

Objects are an instance of a class. It is an entity that has state and behavior. In a nutshell, it is an instance of a class that can access the data.

```
#syntax
class EduClass:

def func(self):
print('Hello')

# create a new EduClass
ob = EduClass()
ob.func()
```

6.2 Abstraction

Abstraction is used to simplify complex reality by modeling classes appropriate to the problem. Here, we have an abstract class that cannot be instantiated. This means you cannot create objects or instances for these classes. It can only be used for inheriting certain functionalities which you call a **base class**. So you can inherit functionalities but at the same time, you cannot create an instance of this particular class. Let's understand the concept of abstract class with an example.

```
from abc import ABC, abstractmethod

class Employee(ABC):
    @abstractmethod

    def calculate_salary(self,sal):
    pass

class Developer(Employee):

    def calculate_salary(self,sal):
        finalsalary= sal*1.10
    return finalsalary

emp_1 = Developer()
print(emp_1.calculate_salary(10000))
#OUTPUT - 11000.0
```

6.3 Inheritance

Inheritance allows us to inherit attributes and methods from the base/parent class. This is useful as we can create sub-classes and get all of the functionality from our parent class. Then we can overwrite and add new functionalities without affecting the parent class. A class that inherits the properties is known as **Child Class** whereas a class whose properties are inherited is known as **Parent class.**

TYPES OF INHERITANCE

- Single Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Multiple Inheritance

```
class employee:
    num_employee=0
    raise_amount=1.04def
__init__(self, first, last, sal):
        self.first=first
        self.last=last
        self.sal=sal
        self.email=first + '.' + last
+ '@company.com'
        employee.num_employee+=1def
fullname (self):
        return '{}
{}'.format(self.first, self.last)
    def apply_raise (self):
        self.sal=int(self.sal *
raise_amount)
class developer(employee):
    pass

emp_1=developer('maxwell', 'sage',
1000000)
print(emp_1.email)
#OUTPUT - maxwell.sage@company.com
```

6.4 Encapsulation

Encapsulation basically means binding up of data in a single class. Python does not have any private keyword, unlike Java. A class shouldn't be directly accessed but be prefixed in an underscore.
Refer to the code on the right-hand side.
Making use of the **setter method** provides indirect access to the private class method. Here I have defined a class employee and used a (__maxearn) which is the setter method used here to store the maximum earning of the employee, and a setter function setmaxearn() which is taking price as the parameter.
This is a clear example of encapsulation where we are restricting the access to the private class method and then use the setter method to grant access.

```
class employee():
def __init__(self):
self.__maxearn = 1000000
def earn(self):
print("earning is:
{}".format(self.__maxearn))

def setmaxearn(self,earn):
#setter method used for accesing
private class
self.__maxearn = earn

emp1 = employee()
emp1.earn()

emp1.__maxearn = 10000
emp1.earn()

emp1.setmaxearn(10000)
emp1.earn()
#earning is:1000000,earning
is:1000000,earning is:10000
```

6.5 Polymorphism

Polymorphism in Computer Science is the ability to present the same interface for different underlying forms. Polymorphism means that if class B inherits from class A, it doesn't have to inherit everything about class A. It can do some of the things that class A does differently. It is most commonly used while dealing with inheritance. Python is implicitly polymorphic, it has the ability to overload standard operators, so that they have appropriate behavior based on their context.

TYPES OF POLYMORPHISM

1. Compile-Time Polymorphism
2. Run-Time Polymorphism

```
class Animal:def __init__(self,name):
        self.name=name
        def talk(self):pass
class Dog(Animal):def talk(self):
                print('Woof')
class Cat(Animal):def talk(self):
        print('MEOW!')
c= Cat('kitty')
c.talk()
d=Dog(Animal)
d.talk()

#OUTPUT - Meow!
#OUTPUT - Woof
```

Chapter 7

PYTHON PROGRAMS FOR PRACTICE

7.1 Reverse a Number using Loop

```
# Get the number from user manually
num = int(input("Enter your favourite number:
"))

# Initiate value to null
test_num = 0

# Check using while loop

while(num>0):
  #Logic
  remainder = num % 10
  test_num = (test_num * 10) + remainder
  num = num//10

# Display the result
print("The reverse number is :
{}".format(test_num))
```

7.2 Fibonacci Sequence using Recursion

```
def FibRecursion(n):
   if n <= 1:
       return n
   else:
       return(FibRecursion(n-1) + FibRecursion(n-
2))
 nterms = int(input("Enter the terms? "))
 #take input from the user

if nterms <= 0:
# check if the number is valid
   print("Please enter a positive integer")
else:
   print("Fibonacci sequence:")
   for i in range(nterms):
       print(FibRecursion(i))
```

7.3 Bubble Sort in Python

```
#repeating loop len(a)(number of elements)
number of times
for j in range(len(a)):
    #initially swapped is false
    swapped = False
    i = 0while i<len(a)-1:
    #comparing the adjacent elements if
a[i]>a[i+1]:

            #swappinga[i],a[i+1] =
a[i+1],a[i]

            #Changing the value of swapped
            swapped = True
        i = i+1

    #if swapped is false then the list is
sorted#we can stop the loopif swapped ==
False:
        break
print (a)
```

7.4 Diamond Pattern with Numbers

```
def pattern(n):
    k = 2 * n - 2
    x = 0
    for i in range(0, n):
        x += 1
        for j in range(0, k):
            print(end=" ")
        k = k - 1
        for j in range(0, i + 1):
            print(x, end=" ")
        print(" ")
    k = n - 2
    x = n + 2
    for i in range(n, -1, -1):
        x -= 1
        for j in range(k, 0, -1):
            print(end=" ")
        k = k + 1
        for j in range(0, i + 1):
            print(x, end=" ")
        print(" ")
pattern(5)
```

PYTHON PRACTICE PROGRAM EXAMPLES

Chapter 8

FREQUENTLY ASKED INTERVIEW QUESTIONS

Today Python has evolved as the most preferred language and considered to be the "**Next Big Thing**" and a "**Must**" for Professionals. This chapter covers the questions which will help you in your Python Interviews and open up various Python career opportunities available for a Python programmer.

1. What type of language is Python?
2. What are the key features of Python?
3. What is the difference between lists and tuples?
4. What are Python modules?
5. What are various built-in data types in Python?
6. What is the difference between arrays and lists?
7. What is __init__?
8. What is a Lambda function?
9. What are the generators in Python?
10. What are docstrings in Python?
11. What is type conversion in Python?
12. How is memory managed in Python?
13. What is a dictionary in Python?
14. What is: *args, **kwargs & why is it used?
15. What is a negative index?
16. What are Python packages?
17. Does Python have OOps concepts?
18. Difference between deep and shallow copy?
19. How is Multithreading achieved in Python?
20. What are Python libraries?
21. What type of language is Python?
22. What is monkey patching in Python?
23. Does python support multiple Inheritance?
24. What is Polymorphism? What are its types?
25. How do you do data Abstraction in Python?
26. Can you create an empty class in Python?
27. WAP in Python to print a Star Pyramid.
28. Explain what Flask is and its benefits?
29. Differentiate between Django, Pyramid and Flask.
30. How you can set up the Database in Django.
31. What are abstract classes in Python?
32. Is Python Numpy better than lists?
33. What is the difference between NumPy and SciPy?
34. What are the limitations of OOPs in Python?
35. Differentiate between Abstraction & Encapsulation.
36. What are pure virtual functions?
37. What is a destructor?
38. What are the types of constructors in Python?
39. What is Garbage Collection(GC)?
40. Differentiate between an error and an exception?

sales@edureka.co

edureka!

IN: 9606058406
US: 18338555775

CAREER GUIDANCE

WHO IS A PYTHON DEVELOPER?
There is no textbook definition for a Python Developer, there are certain domains and job roles a Python Developer can take according to the skill-set they have.

Python Web Developer

A **Python Web Developer** is required to write server side web logic. They should be familiar with web frameworks and HTML and CSS which are the foundation stones for Web Development. Good Database knowledge and writing Python scripts is a nice-to-have skill. Libraries like Tkinter for GUI based Web Applications is a must. Master all these skills and you will become a Python Web Developer.

Software Developer/ Engineer

A **Software Developer/Engineer** must be well-versed with core Python, web frameworks and Object relational mappers. They should have an understanding of multi-process architecture and RESTful API's to integrate applications with other components. Front-end development skills and database knowledge are a few nice-to-have skills for a software developer. Writing Python scripts and system administration is also an add-on when you are aiming to become a Software Developer.

Data Analyst

A **Data Analyst** is required to carry out Data Interpretation and Analysis. They should be well versed with Mathematics and Statistics. Python libraries like Numpy, Pandas, Matplotlib, Seaborn etc., are used for Data Visualization and Manipulation and hence, learning Python can be boon here as well.

Machine Learning Engineer

Machine Learning Engineer must understand the Deep Learning concepts along with Neural-Network architecture and Machine Learning algorithms on top of Mathematics and Statistics. A **Machine Learning Engineer** must be proficient enough in Algorithms like Gradient Descent, Regression Analysis and building Prediction Models.

Data Scientist

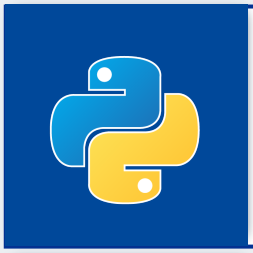
A **Data Scientist** should have thorough knowledge of Data Analysis and Data Interpretation, Data Manipulation, Mathematics and Statistics in order to help in decision making process. They also have to be experts in Machine Learning and AI with all the Machine Learning algorithms like Regression Analysis, Naive-Bayes etc. A **Data Scientist** must know libraries like Tensorflow, Scikit-learn etc., thoroughly. A **Data Scientist** is going to fulfill roles that involves all round development.

EDUREKA PYTHON TRAINING PROGRAMS

PYTHON PROGRAMMING CERTIFICATION

Weekend/Weekday | Live Class | 24 x 7 Technical Assistance

www.edureka.co/python-programming-certification-training



PYTHON TRAINING FOR DATA SCIENCE

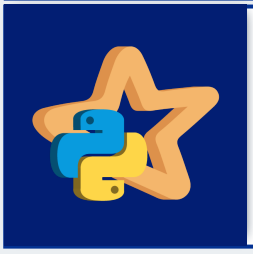
Weekend/Weekday | Live Class | 24 x 7 Technical Assistance

www.edureka.co/data-science-python-certification-course

PYTHON DEVELOPER MASTERS PROGRAM

Weekend/Weekday | Live Class/SP | 24 x 7 Technical Assistance

www.edureka.co/masters-program/python-developer-training



PYSPARK CERTIFICATION TRAINING

Weekend | Live Class | 24 x 7 Technical Assistance

www.edureka.co/pyspark-certification-training

LEARNER'S REVIEWS

S — Shehna

I did the training on **Python for Data Science from Edureka**. Instructor was very knowledgeable. The support team is very responsive. I would definitely recommend **Edureka**

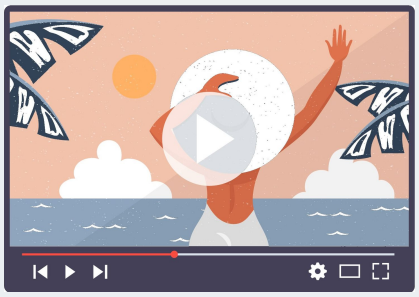
PK — Pawan Kumar

The course content is very good with easily understandable text with examples. The **support team** at back end is excellent as they give full support for any kind of doubts **24*7**.

SN — Swapnil Naresh

The course curriculum was **well structured** and the instructor taught me very well. The query were solved conveniently. Overall great learning and experience.

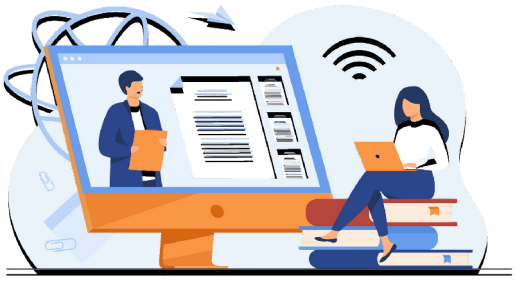
Free Resources



3000+
Video Tutorials on
YouTube

2500+ Technical
Blogs

Active
Community



30+
Free Monthly
Webinars

About Us

There are countless online education marketplaces on the internet. And there's us. We are not the biggest. We are not the cheapest. But we are the fastest growing. We have the highest course completion rate in the industry. We aim to become the largest online learning ecosystem for continuing education, in partnership with corporates and academia. To achieve that we remain ridiculously committed to our students. Be it constant reminders, relentless masters or 24 x 7 online technical support - we will absolutely make sure that you run out of excuses to not complete the course.

Contact Us

IndiQube ETA, 3rd Floor,
No.38/4,
Adjacent to Dell EMC2,
Dodanekundi,
Outer Ring Road, Bengaluru,
Karnataka - 560048

IN: 08035068108 | US: +1415 993 4602
www.instagram.com/edureka.co/
www.facebook.com/edurekaIN
www.linkedin.com/company/edureka/
www.youtube.com/user/edurekaIN
t.me/s/edurekaupdates

News & Media

INDIA TODAY

Edureka partners with **NIT Warangal** to upskill IT professionals in **AI and Machine Learning**

Deloitte.

Edureka (Brain4ce Education Solutions) tops Deloitte Tech Fast 50 2014 rankings

edureka!